

BIZREACH

AI駆動開発が変える、大規模開発の前提

Human in the Loop から Human on the Loop へ

株式会社ビズリーチ CTO 外山英幸

自己紹介



Visionalグループ 株式会社ビズリーチ

執行役員 CTO / DX本部 本部長 / AI Product Studio室 室長

外山 英幸 (Hideyuki Toyama)

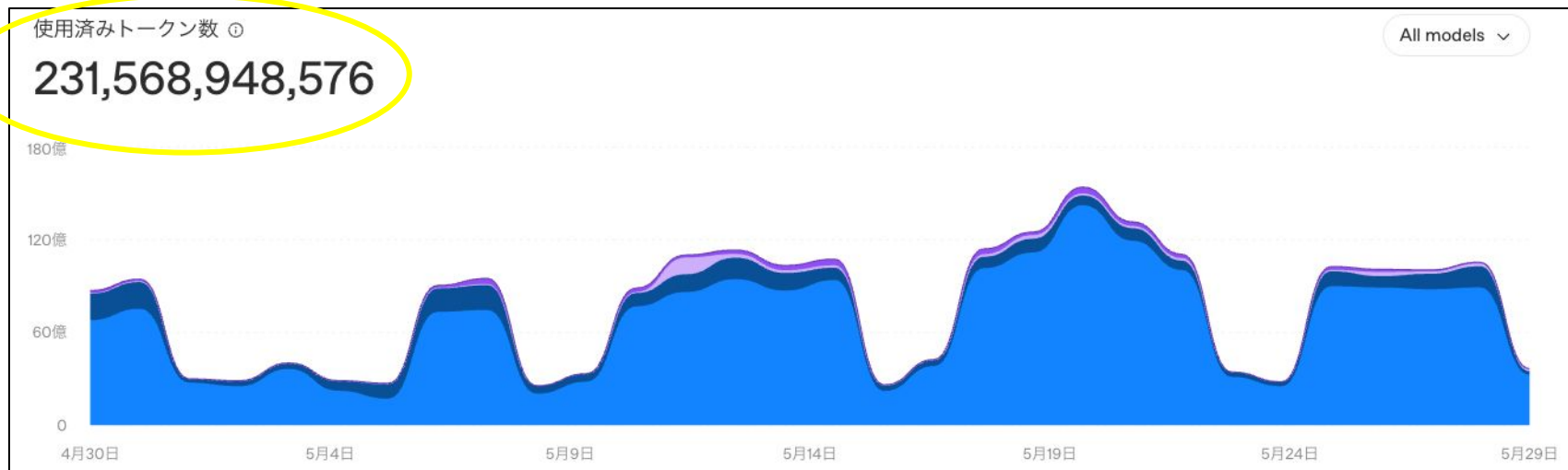
2017年 株式会社ビズリーチ入社

- ・ビズリーチの開発部長などを歴任し、2021年に執行役員 CTOに就任
- ・事業横断のプロダクトロードマップ策定、AI戦略の策定などを担当
- ・DX本部長として、AIを活用した全社の業務プロセス改革も主導

ビズリーチは、Codexで月間2,300億トークンを消費している

総使用量、1アクティブユーザー当たり使用量、共に日本トップクラス。

※Codexのみの数字。Claude Codeやgithub copilot等を含めるとさらに大きい。



2026年 5月実績(350ユーザー)

AI 駆動開発の次に来るのは、何か

AI 駆動開発は、すでに一般化。多くの組織が「もっと使う」フェーズにある。

では、その次に来る課題は何か？



世界はすでに「Harness engineering」に向かっている

2026年2月、業界の議論の中心が変わった。

2月5日 Mitchell Hashimoto氏
(HashiCorp共同創業者)

ブログ『My AI Adoption Journey』で
「Engineer the Harness」を提唱

2月11日 OpenAI (Ryan Lopopolo氏)

『**Harness engineering**: leveraging Codex
in an agent-first world』

→ 3名チーム、5ヶ月、100万行、ゼロ手書き

→ **Harness engineering が、AI Agent 時代の中核的問いに**

harness: モデルを agent として動かす「足場」

違いを生むのは、モデルだけではない。harness の設計も、決定的な差を作る。

harness とは、AIエージェントが 正しく・速く・安全に動くための、

- 実行環境 (execution environment)
- 文脈 (context)
- ガードレール (guardrails)
- 検証ループ (verification loop)
- 観測性 (observability)

を統合して設計する営み

→ モデルだけでなく、harness の設計も、競争領域に

しかし、harness単体では HITL の高速化止まり

harness を真剣に作ると、確かに開発は劇的に速くなる。

しかし、それは「HITL (Human In The Loop) のまま」速くなっただけ。

harness で起きること

- ✓ レビューサイクルが短縮
- ✓ 1人あたりアウトプットが増える
- ✓ 待ち時間が消える
- ✓ デバッグが速くなる

→ 開発速度は劇的に向上

それでも変わらないこと

- ✗ 人間がフロー内部にいる構造
- ✗ レビューが律速になる構造
- ✗ 設計・判断・承認が人間に集中

→ 人間が律速のまま

→ **harness は HITL を高速化するが、HITL を抜け出させない**

問いは「AIをどう使うか」ではない

AI実行を、どう統治するか

Governance of AI Execution

→ ここからが、今日の本題

HITL(Human in the Loop)－現状の構造

人間がフローの「中」にいて、すべての判断点を通過する。



この構造で起きていること

- 設計判断は人間が行う
- 実装レビューは人間が行う
- テスト承認は人間が行う
- リリース判断は人間が行う

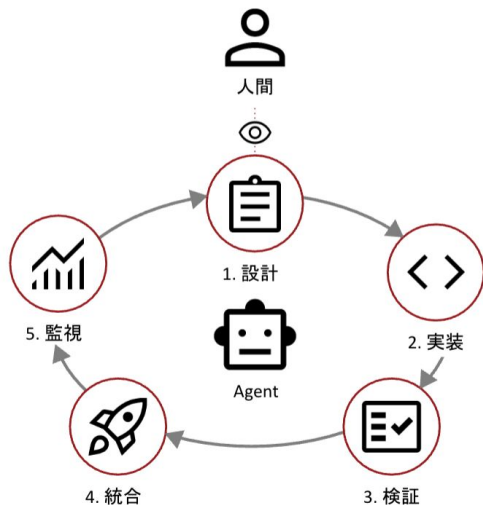
どこかで人間が止まれば、フロー全体が止まる

→ 人間が、すべての律速点になる

HOTL (Human on the Loop) – 目指す構造

人間がループの「外」から監督する。フローには介在しない。

Human **on** the loop



この構造で起きていること

- 設計判断は **Agent** が行う
- 実装は **Agent** が行う
- 検証は **Agent** が行う
- 統合・監視も **Agent** が継続

人間は「監督」と「方向付け」に集中

→ 人間は、律速点から外れる

HITL と HOTL、違いは「制御位置」

違うのは速度ではない。制御位置と、最適化対象が違う。

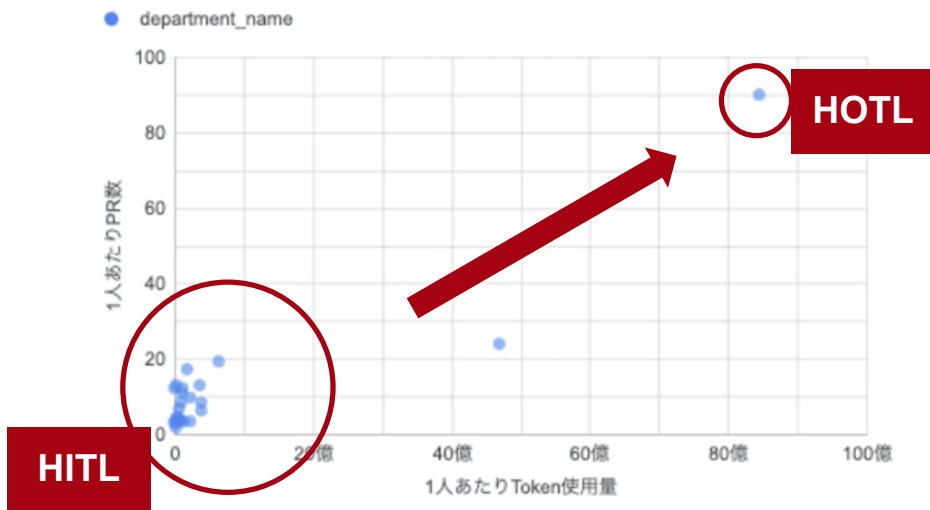
	HITL	HOTL
人間の位置	フロー内部	ループ外部
AIの役割	実装担当	実行・判断主体
律速要因	人間のレビュー	制約・検証の設計
最適化対象	個人	ループ構造

→「人を速くする」のではなく、「ループ構造を設計する」。

実は、社内でも HOTL 型組織だけが、桁違いに突出している

HITL と HOTL の差は、結果として「桁違いの差」になる。

1人あたりのトークン使用量 × PR数 / 月



→「では、なぜここまで差が出るのか？」

ADR を書いても、HOTL にはならない

「書かれている」と「効いている」は、まったく違う。

全部書いてあるのに..

- ✓ ADR: なぜそう設計したか
- ✓ Wiki: 何が正しいか
- ✓ 設計書: 仕様間の依存関係
- ✓ Confluence: ドメインルール

全部、書かれている。

でも、現実には

- ✗ Agent は参照しない
- ✗ 違反しても、誰も止めない
- ✗ 変更影響を、追跡できない
- ✗ 削除していいか、誰も分からない

全員、痛みを知っている。

→ 「書かれている」状態は、HOTL の前提にはなれない

「書かれている」の延長線上に、HOTL はない

ADR を増やしても
設計書を充実させても
Wiki を整備しても
到達しない。

ドキュメントを「書く」のではなく、「ルール」
「チェック」「実行」を
「制御構造に組み込む」必要がある。

→ では、どう効かせるのか？

我々が辿り着いた結論

AI を統治するには、3つの機能を分離する必要がある。

① ルールを、定める

従来

全部、人間(特に熟練者)が持っていた

② ルールに従っているかを、判定する

③ ルールに基づいて、実行する

AI Native (HOTL)

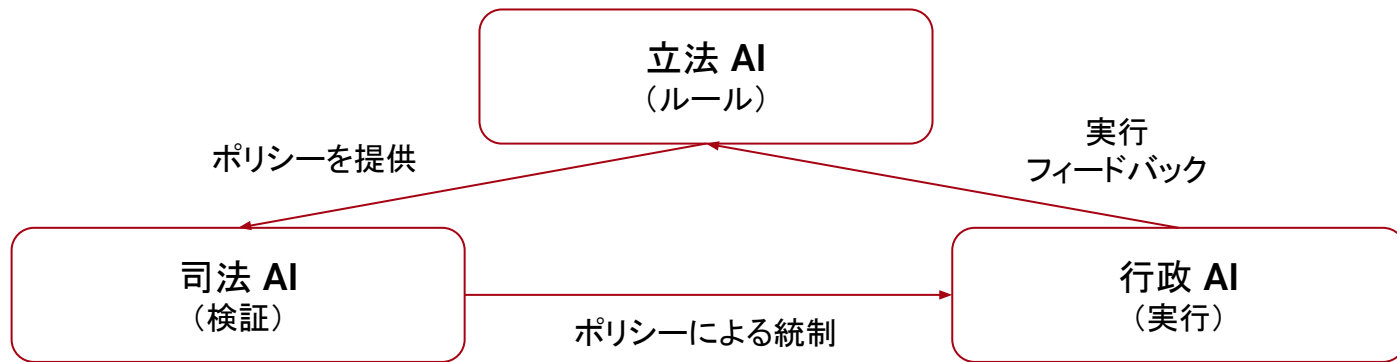
構造として分離する必要がある

→ **この分離が、HOTLに至る前提となった**

AI 駆動開発の統治構造：三権分立

※ ビズリーチ AI Product Studio室 で構想・運用する独自の統治モデル

制御に組み込むために、判断を「立法・司法・行政」に分離する



立法: 何が正しいかを定義する(ルール の SSOT)

司法: 適合しているかを裁く(contracts / guards)

行政: 実際に実行する(workflow / agent)

→ AI が前提の時代の、人と AI の「権力分立」

三権の中身：憲法に基づく立法・司法・行政

憲法が三権を縛る。人が持つ知識を、徹底的に実体化してシステムで縛る。

憲法

(Constitution)

普遍的な開発原則 / 三権全てがこれに準拠 / 改正は人間のみが可能

立法

ルール

憲法に準拠する具体規範

制定/改正

AIが提案、人間が承認

司法

セマンティックチェック

LLMで意味判定

決定性チェック

Grep/AST等で機械判定

行政

ワークフロー

タスク完了の事前定義

スキル+ツール

エージェントを拡張

→ ベテランの頭の中を、三権に分けて「実体化」する

汎用 harness は、三権のどこを担うのか？

立法と司法は、プロダクト・フェーズ依存。

汎用化できるのは harness(行政)だけ。

立法(ルール)

「何が正しいか」は
プロダクトドメイン依存

HR TechとECでは、正しさが違う。
→ 汎用化しづらい

司法(チェック)

「何をチェックすべきか」は
フェーズ依存

初期と大規模では、必要な司法が違う。
→ 汎用化しづらい

行政(harness)

環境分離、context管理、
verification loop

どのプロダクトでも共通
→ 汎用化しやすい

[具体例]

Anthropic、OpenAI、Cursorなど、harness engineeringを提供する側は、構造的に行政までしか触れない。

→ harness は借りられる。統治は、自社で作るしかない。

Authority Provenance Graph : ルールの由来を追跡する graph

我々がやりたかったのは graph を作ることではない。

「書かれている」を「効いている」に変えること。

立法(ルール)と司法(チェック)を、
機械可読に・双方向に接続する層。

例:

- ADR のルール ↔ それを検証するテスト
- ドメインルール ↔ 違反検知の **assertion**
- 設計原則 ↔ **Lint**er ルール

※ 内容を書くのではなく、接続を担う層

①立法なき司法

チェックは存在するが、それが何のルールに依拠するか不明

②司法なき立法

ルールは存在するが、それを効かせるチェックがない

③越境司法

チェックが、本来管轄外の領域まで裁いている

→ graph があれば、これら3つを機械可読に検知できる

Specification Provenance Graph : 機能の成立を追跡する graph

この機能は、何の仕様にに基づき、どのテストで証明されているか。

それを機械可読に追跡する。



「この機能は何の仕様にに基づくか？」

「どのテストで証明されているか？」

「変更したら、何に影響するか？」

Authority Provenance Graph × Specification Provenance Graph

2つの graph が揃って、初めて「機能・非機能の統治」が完成する

→ Agent が自走するために必要な「追跡可能性」を機械可読に

graph 設計の最重要原則 – SSOT と派生データの分離

自動生成される情報を、判断の根拠として参照させてはならない。

SSOT (Single Source of Truth)

「人が定義したもの」
「正式な承認プロセスを経たもの」

例:

- 機能要件
- ドメインルール
- テスト・assertion

→ 判断の根拠として参照可能
→ 変更には承認が必要

派生データ

「機械が自動生成するもの」
「いつでも再生成できるもの」

例:

- 変更影響分析
- 自動生成された依存図
- 集計サマリー

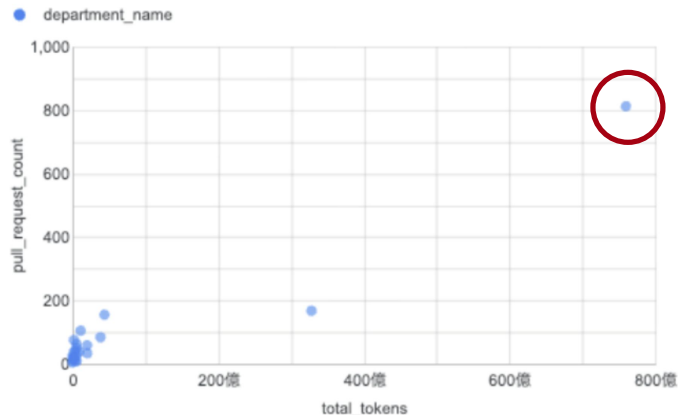
→ 判断の根拠としては参照不可
→ ナビゲーション・索引としてのみ使う

→ 「便利だから」で SSOT と派生データを混ぜると、統治は必ず崩壊する

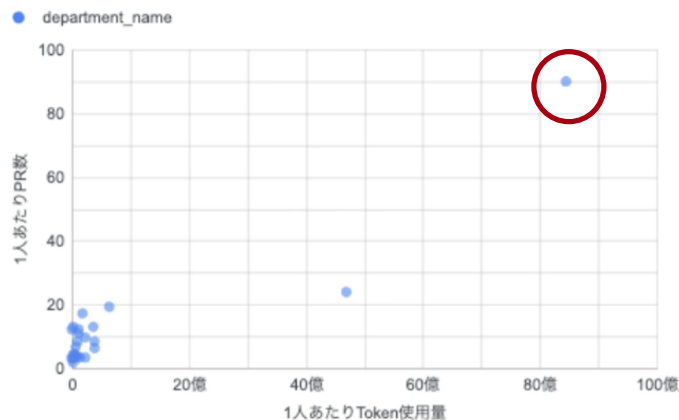
実証: HOTL 型組織だけが、桁違いに突出している

チーム全体でも、一人あたりで見ても、外れ値。これは「規模の差」ではない。

PR数 × Codextトークン消費量 / 月
(チーム全体)



PR数 × Codextトークン消費量 / 月
(一人当たり)



→ 同じ会社、同じツール、同じ時代。それでも、桁違いに差が出る。

違いを生んだのは「AI 利用量」ではない

違いを生んだのは、AI を「どれだけ使ったか」ではない。

AI を「どれだけ自律的に走らせられる構造を持っているか」である。

AI 実行密度

AI Agent が、人間の介入なしに、自律的に走り続けられる「密度」

→ AI を使ったのではない。AI を前提に、組織構造を変えた。

harness は必要条件、統治は十分条件

harness だけでは HOTL に到達しない。統治が、harness を動かす。

harness (必要条件)

業界共通

- 環境、文脈、制約、検証ループ、観測性
- AI Agent が動ける基盤
- 業界中立で提供できる
- 借りられる

統治 (十分条件)

ビズリーチ独自

- 三権分立モデル
- Authority Provenance Graph
- Specification Provenance Graph
- AI Agent を統治する層、
- プロダクト・組織依存

現状は自社で作る
しかない

→ **harness だけでは HOTL に到達しない。統治で、初めて完成する。**

HOTL化に向けて、明日から自社で問えること

問うべきは「書かれているか」ではない。「効いているか」である。

① 判断の根拠は、機械的にチェック可能な形で表現されているか？

→ ドキュメントに「ある」だけでなく、違反したらCI が落ちる、Agent が止まる状態か？

→ ADR / 設計書 / Wiki は、Level 1 に過ぎない

② 派生情報と原典は、機械的に区別されているか？

→ 自動生成された情報が、判断根拠として参照可能になっていないか？

→ 「便利だから」で、原典と派生が混在していないか？

③ ルール・検証・実行の繋がりは、Agentが追跡し、Agentが止められるか？

→ 人間が記憶や経験で繋いでいる状態になっていないか？

→ 接続が切れたら、Agent が違反を検知できるか？

→ 「書かれている」で Yes にしてはいけない。「効いている」かを問え。

我々が今、向き合っている問い

今日話した統治は、HOTL の入口に過ぎない。

HOTL は「AI 前提の世界」、人間 First の延長線上にはない。

機能以外の領域に、どう統治を広げるか

- セキュリティ、堅牢性、保守品質
- インフラ、運用、DevOps(挑戦中)

graph 設計を、どう深めるか

- 直交 linkage の活用
- 状況に応じたナレッジ graph 化
- weight 設計、その他

▼
突き詰める覚悟が必要

プロダクトを、AI-Native 前提で再設計するとは？

- 「人間 First」ではないドキュメント設計
- HOTL を前提とした機能設計

組織・プロセスを、HOTL 前提で再設計するとは？

必要な人材要件は、どうアップデートされるか？

▼
変革の意思が必要

→ 我々もまだ挑戦中。これら全ては、AI-Native時代の組織が向き合う問い。

AI Building AI の時代、競争力を決めるのは

AI を使う量ではない。

AI 実行を、統治できるかどうか。

そしてこれは、開発の話に見えて、
事業と組織のあり方 の話に接続する。

BizReach AI Open Sessions in BIZCON 2026

7/1-7/2 オンライン配信(無料)

- プロダクトをどう作り変えるか
- 提供価値はどう変わるか
- 組織変革をどう進めるか



OpenAI様やGoogle様
とも対談します

▼興味のある方はこちら



仲間募集中：AI Product Studio室

「Yesterday is Dead.」 今日話した内容も、来月にはレガシー。

AI Product Studio室は、HOTL を実装している「特務チーム」です。

常に「変わり続けるために、学び続ける」組織です。

今は求めている人

✗これから挑戦しようとしている人

→ 社内への機会提供を優先したい

求めている人

✓すでに個人で走り始めている人

✓自分なりの考えを持っている人

✓毎日ディスカッションし、集合知として成長させ
続けたい人

✓価値を生み出すために、統治を行いたい人

→ **「Yesterday is Dead.」 一緒に、今日を作りませんか。**

ご清聴、ありがとうございました。

AIカンパニーへの変革をリードする
企業の現在地と未来

BizReach AI[◆]
Open Sessions in BIZCON 2026

7.1 Wed. 2 Thu.

オンライン無料配信

